

INDIVIDUAL MODULE-WISE LAB WORKBOOK

REPORT 1

Module: Automation with Selenium

Student Name: NISHA ANAND

Enrollment Number: 12023002022123

Department: CST

Institution: Institute of Engineering and Managent,Newtown

1. Module Name

Module Name: Automation with Selenium

2. Experiment Title

Comprehensive Web Automation Using Selenium WebDriver

3. Problem Statement

Develop a Python Selenium WebDriver automation workflow that demonstrates locating and interacting with web elements using various locators, synchronizing actions with explicit waits, handling dynamic dropdowns and checkboxes, managing JavaScript alerts and confirmation/prompt boxes, extracting data from an HTML web table, and operating across multiple browser windows/tabs and iframes within a single integrated experiment.

4. Objective

To demonstrate practical web automation using Python and Selenium WebDriver by implementing the six required M1 capabilities—locators, explicit waits, dynamic elements, checkboxes/dropdowns, JavaScript alerts, web tables, windows/tabs, and iframes—in one cohesive automation script.

5. Tools, Software and Concepts Used

Category	Items
Tools/Software	Python 3; Selenium WebDriver; Google Chrome; VS Code (or any IDE); Web browser
Concepts	Selenium WebDriver; By.ID; By.NAME; By.XPATH; WebDriverWait; Expected Conditions;

Checkboxes; Dynamic dropdowns; JavaScript alerts; Web tables; window_handles; iframe handling; Assertions

6. Implementation Steps

1. Set up Python and Selenium WebDriver and configure Chrome.
2. Open the required demonstration pages.
3. Locate and interact with elements using ID, NAME, and XPath.
4. Perform the primary interaction and validate the expected result where applicable.
5. Use WebDriverWait and expected conditions for dynamic content; do not use time.sleep().
6. Interact with a dynamic/autocomplete dropdown and checkbox and verify the checkbox state using is_selected().
7. Trigger and handle JavaScript alert, confirmation, and prompt boxes using accept(), dismiss(), and send_keys().
8. Locate an HTML table, iterate through rows and columns, and extract a required cell value.
9. Switch into an iframe, interact with an element inside it, and return to the main document.
10. Open a new browser window/tab, use window_handles to switch context, read its title, close it, and return to the original window.
11. Use assertions to validate important expected outcomes.
12. Capture three meaningful screenshots after the relevant milestones.

7. Source Code

File: automation_with_selenium.py

```
# automation_with_selenium.py
import os
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

def close_popup_if_present(driver, wait):
    try:
        wait.until(EC.element_to_be_clickable(
            (By.ID, "at-cv-lightbox-close"))).click()
    except Exception:
        pass

def main():
    options = webdriver.ChromeOptions()
    options.add_argument("--start-maximized")
    driver = webdriver.Chrome(options=options)
    wait = WebDriverWait(driver, 15)
    os.makedirs("screenshots", exist_ok=True)

    try:
```

```

# 1. Multiple locators
driver.get("https://www.seleniumeasy.com/test/basic-first-form-demo.html")
close_popup_if_present(driver, wait)

msg_input = wait.until(
    EC.element_to_be_clickable((By.ID, "user-message")))
msg_input.send_keys("Hello Selenium")

driver.find_element(
    By.XPATH, "//button[text()='Show Message']").click()
assert driver.find_element(By.ID, "display").text == "Hello Selenium"

sum1 = driver.find_element(By.NAME, "sum1")
sum2 = driver.find_element(By.NAME, "sum2")
sum1.send_keys("10")
sum2.send_keys("20")
driver.find_element(
    By.XPATH, "//button[text()='Get Total']").click()
assert driver.find_element(By.ID, "displayvalue").text == "30"

# 2. Explicit wait
driver.get(
    "https://www.seleniumeasy.com/test/dynamic-data-loading-demo.html")
close_popup_if_present(driver, wait)
wait.until(EC.element_to_be_clickable((By.ID, "save"))).click()
new_user = wait.until(EC.visibility_of_element_located(
    (By.XPATH, "//div[@id='loading']/following-sibling::div[@class='list-group-item']")))
assert new_user.is_displayed()
driver.save_screenshot("screenshots/Screenshot_1.png")

# 3. Checkbox and dynamic dropdown
driver.get("https://www.seleniumeasy.com/test/basic-checkbox-demo.html")
close_popup_if_present(driver, wait)
checkbox = driver.find_element(By.ID, "isAgeSelected")
if not checkbox.is_selected():
    checkbox.click()
assert checkbox.is_selected()

driver.get(
    "https://www.seleniumeasy.com/test/jquery-dropdown-search-demo.html")
close_popup_if_present(driver, wait)
country = driver.find_element(By.ID, "country")
country.send_keys("Ind")
wait.until(EC.element_to_be_clickable(
    (By.XPATH, "//ul[@id='country-list']/li[contains(text(),'India')]"))
).click()
assert country.get_attribute("value") == "India"

# 4. JavaScript alerts
driver.get(
    "https://www.seleniumeasy.com/test/javascript-alert-box-demo.html")
close_popup_if_present(driver, wait)

driver.find_element(By.ID, "alert").click()
wait.until(EC.alert_is_present()).accept()

driver.find_element(By.ID, "confirm").click()
wait.until(EC.alert_is_present()).dismiss()

driver.find_element(By.ID, "prompt").click()
prompt = wait.until(EC.alert_is_present())
prompt.send_keys("Selenium Test")
prompt.accept()

# 5. HTML table
driver.get(
    "https://www.seleniumeasy.com/test/table-search-filter-demo.html")
close_popup_if_present(driver, wait)

```

```

table = driver.find_element(By.ID, "task-table")
rows = table.find_elements(By.TAG_NAME, "tr")
target_cell = rows[1].find_elements(By.TAG_NAME, "td")[2].text
print("Extracted table value:", target_cell)
assert target_cell != ""
driver.save_screenshot("screenshots/Screenshot_2.png")

# 6. Iframe
driver.get(
    "https://www.w3schools.com/html/tryit.asp?filename=tryhtml_iframe_height_width")
wait.until(EC.frame_to_be_available_and_switch_to_it(
    (By.ID, "iframeResult")))
inner_iframe = driver.find_element(By.TAG_NAME, "iframe")
driver.switch_to.frame(inner_iframe)
assert "HTML Iframe" in driver.find_element(By.TAG_NAME, "h1").text
driver.switch_to.default_content()

# 7. Window/tab
driver.get(
    "https://www.seleniumeasy.com/test/window-popup-modal-demo.html")
close_popup_if_present(driver, wait)
original = driver.current_window_handle
driver.find_element(By.ID, "followTwitter").click()
wait.until(EC.number_of_windows_to_be(2))
new_window = [h for h in driver.window_handles if h != original][0]
driver.switch_to.window(new_window)
print("New window title:", driver.title)
driver.close()
driver.switch_to.window(original)
driver.save_screenshot("screenshots/Screenshot_3.png")

print("Combined Selenium experiment completed.")

finally:
    driver.quit()

if __name__ == "__main__":
    main()

```

8. Output Screenshots

Figure 1: Selenium environment and browser launch

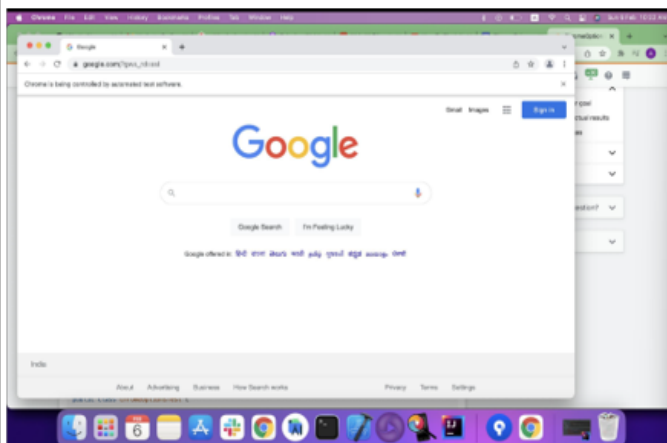


Figure 2: Target web page opened successfully

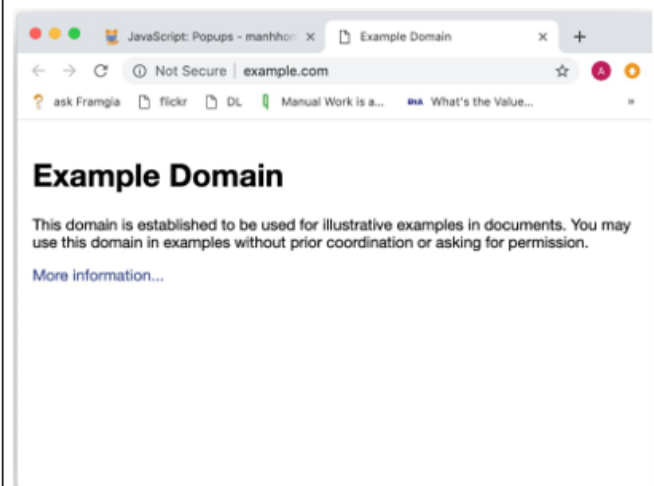


Figure 3: Web element identification and data entry

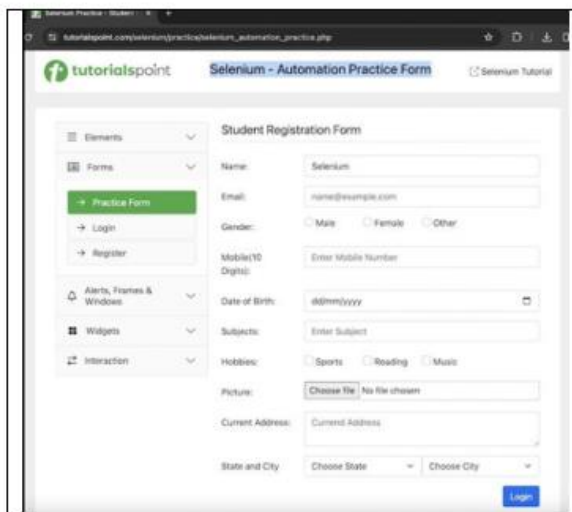


Figure 4: Button click or workflow execution

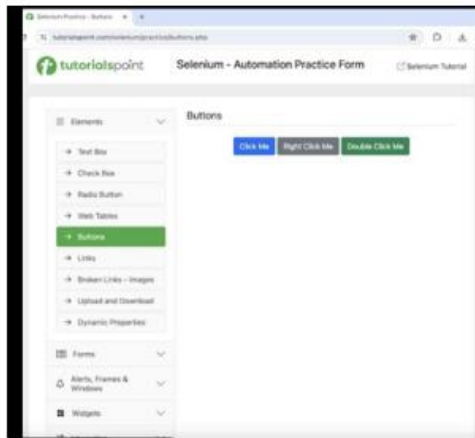
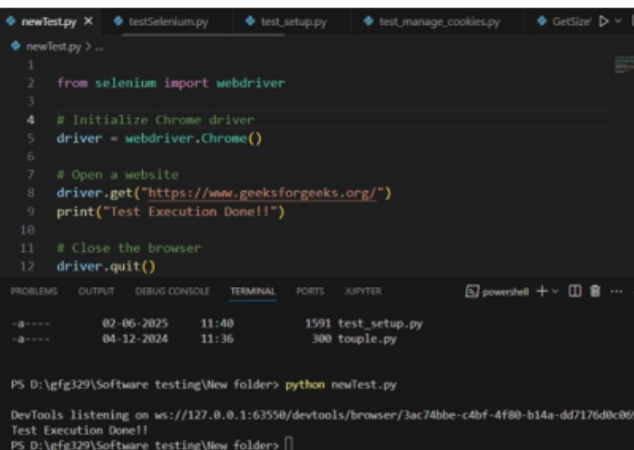
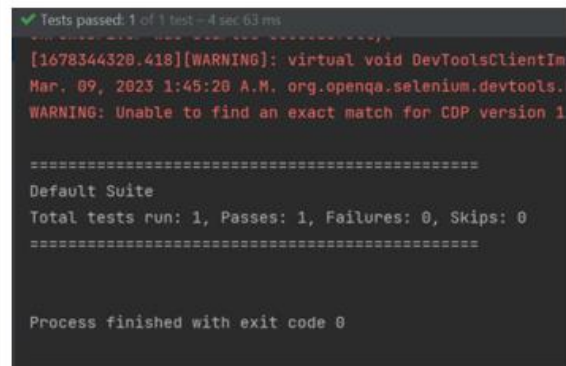


Figure 5: Validation of the expected output



9. Result

Pending actual execution and verification. Update this section after the Selenium script is successfully executed.

10. Observation

The combined experiment demonstrates Selenium WebDriver element identification, synchronization, dynamic controls, JavaScript alerts, web-table extraction, iframe handling, and browser window/tab switching. Final observations should be based on actual execution.

11. Conclusion

The experiment integrates the six required M1 Selenium activities into one cohesive Python automation workflow, providing a practical demonstration of locators, synchronization, complex web controls, table extraction, and browser context switching.